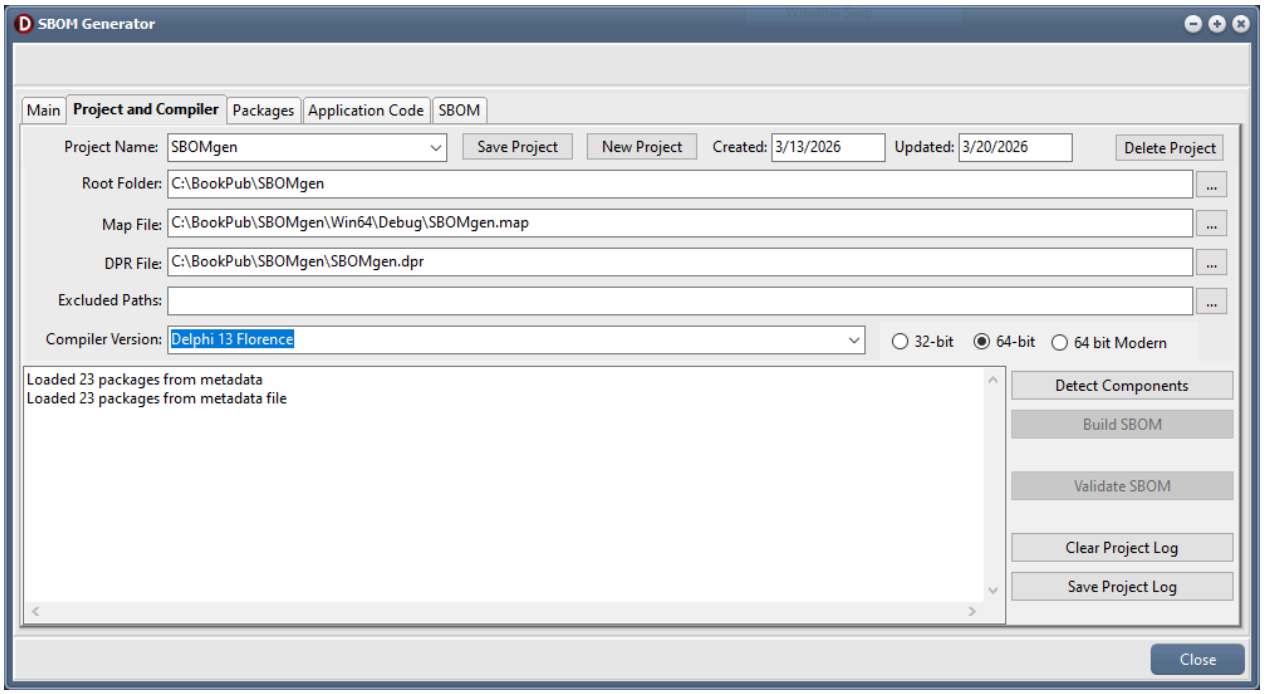
SBOMgen User Manual

William Meyer

2026-03-30

1. SBOMgen User Manual

SBOMgen — Delphi SBOM Generator Version 1.0



1.1. Contents

1. What is an SBOM?
2. CycloneDX and SPDX
3. The Delphi Challenge
4. Getting Started
5. Project Setup
6. The Packages Tab
7. Completing Metadata for Your Own Code
8. The AppCode Tab
9. Generating the SBOM
10. Validating the SBOM
11. The Metadata Catalog
12. Managing Projects
13. Related Tools
14. Troubleshooting

1.2. What is an SBOM?

A Software Bill of Materials (SBOM) is a structured inventory of every component that makes up a software product — third-party libraries, frameworks, and operating system dependencies, along with their versions and licensing terms.

The concept is straightforward: just as a manufactured product has a bill of materials listing every part, a software product should be able to declare what it is made of. This matters for several reasons.

Security. When a vulnerability is discovered in a library — OpenSSL, Log4j, or any other widely-used component — organisations need to know quickly whether their products are affected. An SBOM makes that question answerable in minutes rather than days.

License compliance. Commercial software incorporates third-party components under a variety of licenses. Some licenses are permissive, some require source disclosure, and some prohibit commercial use. An SBOM makes it possible to audit license obligations systematically rather than relying on memory or scattered documentation.

Supply chain transparency. Governments and large enterprises increasingly require suppliers to provide SBOMs alongside delivered software.

An SBOM is not a security audit, a vulnerability scan, or a license analysis tool on its own. It is the foundation that makes those activities possible and repeatable.

1.2.1. The Legislative Context

SBOM requirements have moved from voluntary best practice to legal obligation in several jurisdictions.

United States — Executive Order 14028 (May 2021) directed federal agencies to require SBOMs from software suppliers. The National Telecommunications and Information Administration (NTIA) subsequently published the minimum elements a conforming SBOM must contain: supplier name, component name, version, unique identifier, dependency relationships, author, and timestamp. Details are at https://www.ntia.gov/sbom.

European Union — Cyber Resilience Act (EU) 2024/2847 entered into force on 10 December 2024. This regulation applies to any software product placed on the EU market. Key compliance dates:

Date	Obligation
11 September 2026	Vulnerability and incident reporting obligations begin
11 December 2027	Full CRA compliance mandatory for all products on the EU market

The CRA requires a machine-readable SBOM covering at least the top-level dependencies of each software release, in CycloneDX or SPDX format. You do not submit the SBOM anywhere — you

generate it per release, archive it for at least ten years, and make it available only if a market surveillance authority formally requests it. The official regulation text is at https://eur-lex.europa.eu/eli/reg/2024/2847/oj/eng.

> **Note:** The above is informational. Legislation can be amended, jurisdictions differ, and CRA obligations outside the SBOM (secure-by-design, vulnerability management, incident reporting) are beyond the scope of SBOMgen. Verify current requirements for your specific situation with appropriate legal guidance.

1.3. CycloneDX and SPDX

Two standards dominate the SBOM landscape: **CycloneDX** and **SPDX**. SBOMgen produces CycloneDX output and uses SPDX license identifiers within it.

1.3.1. CycloneDX

CycloneDX is a lightweight SBOM standard originally developed by OWASP (Open Worldwide Application Security Project https://owasp.org/) and now maintained by the CycloneDX community. SBOMgen produces **CycloneDX 1.6** output — also published as **ECMA-424, 1st Edition** — which is the current version of the specification and is accepted for EU CRA compliance. The CycloneDX specification is available at https://cyclonedx.org.

A CycloneDX document contains:

- A **metadata** section identifying the application being described, the tool that generated the SBOM, and a timestamp.
- A **components** array listing each dependency with its name, version, supplier, license, and a unique `bom-ref` identifier.
- An optional **dependencies** section expressing which components depend on which others.

SBOMgen's output has been validated against the official CycloneDX 1.6 schema using the CycloneDX CLI validator with strict validation enabled.

1.3.2. SPDX License Identifiers

SPDX (Software Package Data Exchange) maintains a standard list of license identifiers — short, unambiguous strings that identify specific licenses. Examples include `MIT`, `Apache-2.0`, `GPL-3.0-only`, and `MPL-1.1`. SBOMgen uses SPDX identifiers in the `license` field of each component.

The full SPDX license list is at https://spdx.org/licenses.

When a license is genuinely unknown, SBOMgen uses `NOASSERTION`, which is the standard value for "we cannot determine the license". Components marked `NOASSERTION` should be investigated and resolved before the SBOM is considered complete.

1.3.3. LicenseRef- Expressions

When a license cannot be expressed as a standard SPDX identifier, SBOMgen supports `LicenseRef-` prefixed identifiers. These are valid under both the CycloneDX and SPDX specifications and are the correct way to represent proprietary or non-standard licenses.

Common examples:

- `LicenseRef-Proprietary` — a generic proprietary license
- `LicenseRef-Embarcadero-Proprietary` — Embarcadero's commercial license
- `LicenseRef-MyCompany-Commercial` — a custom commercial license specific to a vendor

The `LicenseRef-` prefix signals to consuming tools that the license is not in the SPDX list but is documented elsewhere. It is preferable to leaving the field empty or using a plain text description, both of which will fail schema validation.

1.3.4. License Expressions

SPDX supports compound license expressions for components distributed under multiple licenses. The operators are `OR` (choose either), `AND` (both apply), and `WITH` (license with exception). For example:

- `MIT OR Apache-2.0` — the recipient may choose either license
- `LGPL-2.1-only WITH Classpath-exception-2.0` — LGPL with a specific exception

SBOMgen handles compound expressions correctly in the generated JSON output, routing them to the `expression` field as required by the CycloneDX 1.6 schema.

1.4. The Delphi Challenge

Most SBOM tooling is designed for ecosystems that use package managers — npm, Maven, NuGet, pip. In those ecosystems, the dependency graph is explicit: a manifest file lists every package by name and version, and a tool can read that manifest and produce an SBOM in seconds.

Delphi does not have a centralised package manager in the same tradition. Dependencies are managed through a combination of:

- The **IDE library paths** configured in Tools → Options
- **GetIt packages** installed into the CatalogRepository
- **Source code** added directly to a project's search path
- **BPL runtime packages** registered in the Windows registry

None of these mechanisms produces a manifest that lists third-party components with their versions and licenses. The Delphi compiler resolves units at build time from wherever the search paths point, but it does not record which units came from which library.

SBOMgen approaches this problem by analysing the **MAP file** produced by the Delphi linker. A MAP file lists every unit that was linked into the final executable. By cross-referencing that unit list against the IDE library paths (to identify bundled Delphi components), the GetIt CatalogRepository (for GetIt-installed packages), the project's own source folders (to identify internal units), and a maintained metadata catalog (to supply supplier, license, and version information), SBOMgen reconstructs the dependency inventory that a package manager would have provided automatically.

The metadata catalog is central to this approach. For components that SBOMgen recognises — the Delphi RTL, VCL, Indy, Spring4D, and many others — metadata is built in. For components specific to your project, you supply the metadata through the editing interface. This extra step is the price of working without a package manager, and it is a one-time effort — once a component is cataloged, every future project that uses it is resolved automatically.

1.5. Getting Started

1.5.1. Prerequisites

- Delphi XE or later. Delphi XE4 and later are confirmed working. Earlier versions including Delphi XE, XE2, XE3, and pre-XE versions have not been fully tested — community feedback on older compiler support is welcome.
- A Delphi project with MAP file generation enabled (see below).
- Optionally, the CycloneDX CLI tool for SBOM validation (see Validating the SBOM).

> **Legacy projects.** Many Delphi codebases being maintained today were originally written in Delphi 2007, 2009, or XE-series compilers. SBOMgen is designed with these projects in mind — generating an SBOM for a legacy codebase is often more urgent than for a new one. If you encounter issues with a specific older compiler version, please report them via the project repository.

1.5.2. Enabling MAP File Generation

SBOMgen analyses your project's MAP file. MAP file generation is disabled by default in Delphi. To enable it:

1. Open your project in the Delphi IDE.
2. Go to **Project → Options → Delphi Compiler → Linking**.
3. Set **Map file** to **Segments** or **Detailed**.
4. Rebuild the project.

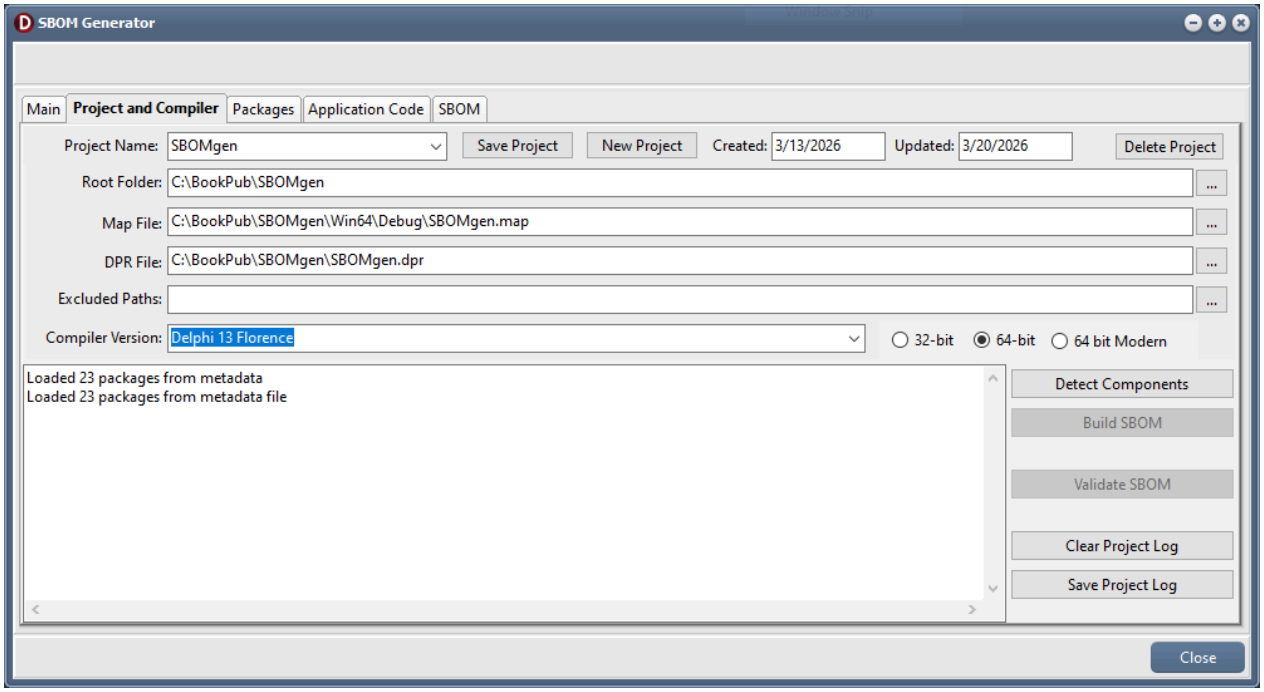
The MAP file will be generated in the same folder as the executable, with a `.map` extension.

1.5.3. Installation

SBOMgen is a standalone executable — no installation is required. Place `SBOMgen.exe` in a convenient location and run it directly.

1.6. Project Setup

When you first run SBOMgen, the project list will be empty. Click **New Project** to create your first project.



1.6.1. Project Name

Enter a descriptive name for the project. This name is used in the generated SBOM as the application name. You will likely wish to use the name of your project, but may also add a suffix, as appropriate. This is purely a string; non-alphanumeric characters are acceptable in the project name.

1.6.2. Delphi Version

Select the version of Delphi used to build your application. SBOMgen uses this to locate the correct IDE library paths, which it needs to identify which units belong to the Delphi runtime.

> Note that the Delphi versions displayed will include only those which are installed in this environment. SBOMgen needs access to the installed Delphi in order to fully document the SBOM.

Each project stores its own Delphi version selection independently. If you maintain multiple projects built with different compiler versions — for example, a legacy product still on Delphi 2007 and a newer product on Delphi 12 — each project will correctly use its own compiler's library paths.

If your installed version does not appear in the list, it may not have been detected in the registry. See Troubleshooting.

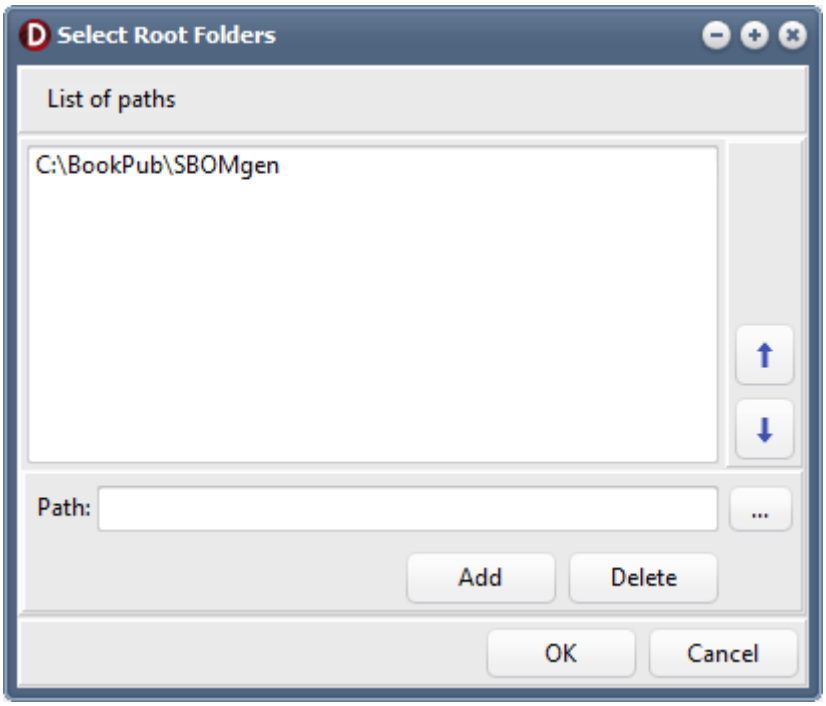
1.6.3. Root Folders

Root folders are the directories that contain your application's own source code. SBOMgen uses these to distinguish your code from third-party dependencies — any unit whose source file is found under a root folder is treated as internal to your application and excluded from the SBOM component list.

Enter one or more folders separated by semicolons:

```
C:\Projects\MyApp\Source;C:\Projects\SharedLibrary\Source
```

You can also use the folder selection button to build the list interactively.



> **Getting this right is important.** If a root folder is missing, units from your own code may be incorrectly identified as external packages. Your project may span multiple source trees — a common example is when application code and shared utility code sit in sibling folders. Both must be listed to ensure all internal modules are correctly identified. If you see your application name appearing as an unknown component in the Packages tab, a root folder is almost certainly missing.

After changing root folders, click **Detect Components** again to reclassify modules with the updated settings.

1.6.4. MAP File

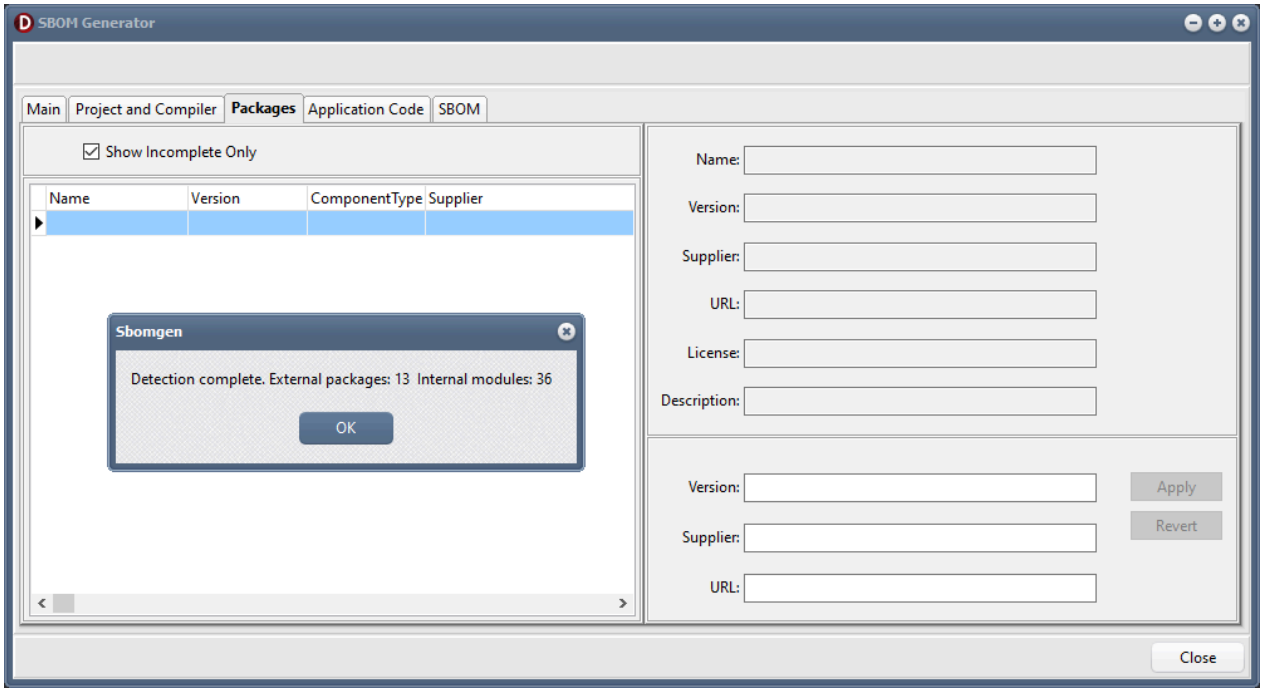
Select the `.map` file generated when you built your project. This is usually in the same folder as the compiled executable.

1.6.5. DPR File

Optionally select your project's `.dpr` file. SBOMgen uses this to read the project's uses clause and can provide additional context for component detection. This field is not required — if left empty, SBOMgen will work from the MAP file alone.

1.7. The Packages Tab

After configuring your project and clicking **Detect**, SBOMgen analyses the MAP file and presents the detected components in the **Packages** tab.



SBOMgen reports the packages detected – external and internal – in the dialog message.

1.7.1. What SBOMgen Detects

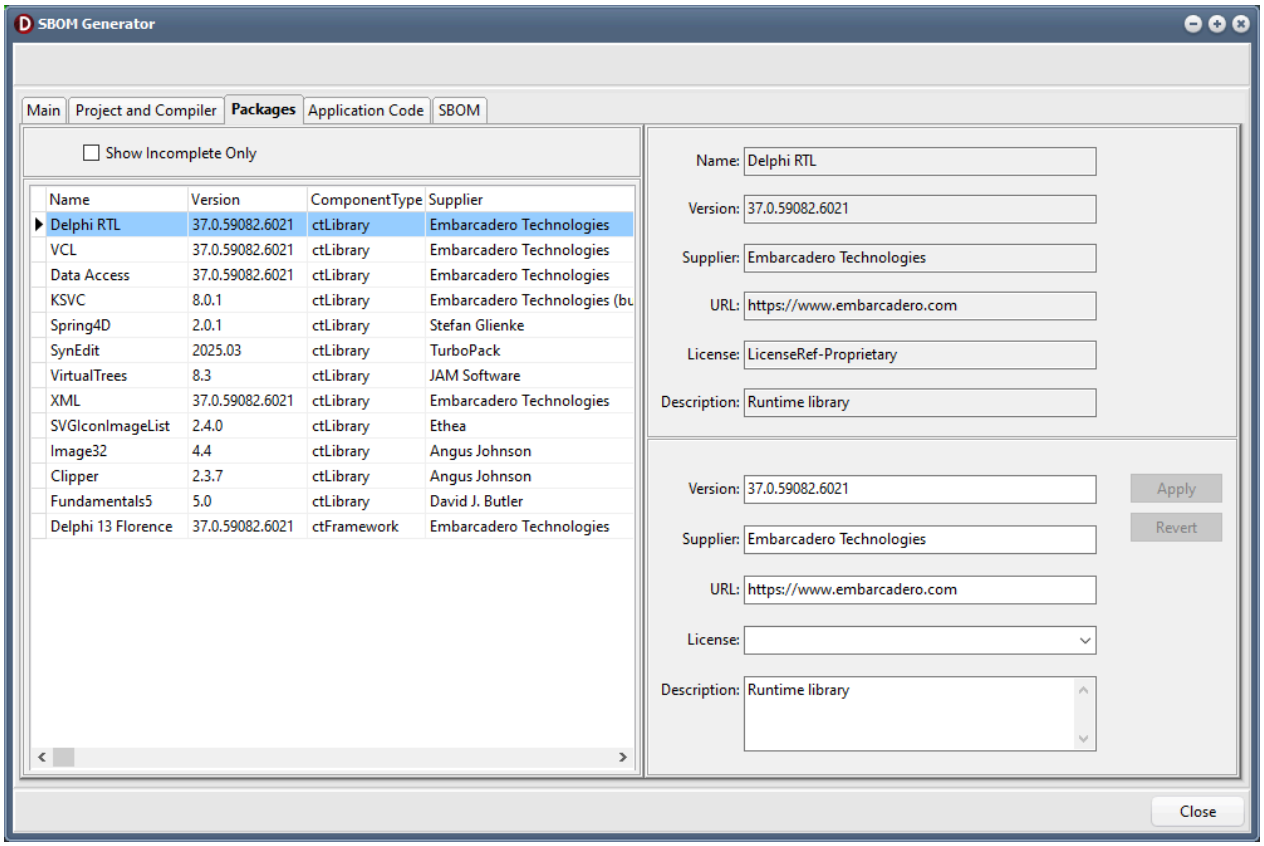
SBOMgen identifies each external component as a package — a named group of units that belong to the same library or framework. The detection process works in three stages:

1. **Delphi built-in components** — units from the RTL, VCL, FireDAC, Indy, XML, SOAP, and other Delphi-bundled libraries are identified from the IDE library paths.
2. **GetIt packages** — units from packages installed via GetIt are identified from the CatalogRepository folder, typically `C:\Users\Public\Documents\Embarcadero\Studio\<version>\CatalogRepository`.
3. **Catalog matching** — remaining units are matched against the built-in metadata catalog using unit name membership lists and prefix rules. Components like Spring4D, SynEdit, VirtualTreeView, Fundamentals5, and others are recognised this way.

Units that cannot be attributed to any known package are grouped by their common prefix and presented as unknown components requiring your attention.

1.7.2. The Package List

Each row in the Packages tab shows the component name, version, supplier, license, and a status indicator. Entries with a warning icon have missing or placeholder metadata. Entries shown in bold have been modified by you and are protected from automatic catalog updates.



The upper right area presents the details for the selected row. The lower right offers editing, and if you select multiple rows, will apply the changes entered to each of them.

1.7.3. Completing Incomplete Entries

Select a component and use the editor panel on the right to fill in missing fields.

The **License** field uses a dropdown list of common SPDX identifiers. If the license you need is not in the list, you can type a `LicenseRef-` prefixed identifier directly — see LicenseRef- Expressions above.

> **If the License dropdown is empty after selecting a component**, the stored license value is not a recognised SPDX identifier. Select a valid license from the list, or type a `LicenseRef-` expression. Do not leave the field empty or set to a plain text description — it will cause validation failures.

Click **Apply** to save changes. Your changes are saved to the global metadata catalog and will be remembered for future projects that use the same component.

1.7.4. Component Versions Across Projects

The metadata catalog stores one entry per component name. If you maintain two projects that use different versions of the same component — for example, DevExpress 22.1 in one project and

DevExpress 23.2 in another — you will need to verify and update the version field in the Packages tab after switching projects.

Two workarounds are available:

- **Edit the version after switching** — select the component in the Packages tab and update the version field before generating the SBOM. This is quick but requires attention each time you switch.
- **Use distinct catalog names** — name the entries `DevExpress-22.1` and `DevExpress-23.2` in the catalog. This preserves both versions permanently but means the component will appear as unknown until you select the correct named entry.

Per-project version overrides are planned as a future enhancement.

1.7.5. Selecting Multiple Packages

Hold Ctrl and click to select additional rows for bulk editing. The selection count is shown in the editor panel. When multiple rows are selected, the editor enters multi-edit mode — only fields you fill in will be applied, leaving other fields unchanged.

> **Note:** Deselecting individual rows with Ctrl-click may behave unexpectedly due to a limitation in the underlying grid component. If the selection becomes inconsistent, click any single row without Ctrl to reset to a single selection, then rebuild the multi-selection from there.

1.8. Completing Metadata for Your Own Code

One situation that surprises new users is seeing their own application appearing as an unknown component in the Packages tab. This happens when some of the application's source files are not under any declared root folder, causing SBOMgen to treat those units as external.

The correct resolution is to add the missing source folder to Root Folders in the project settings, then click Detect again. Once the source files are found under a root folder, those units will be classified as internal and will no longer appear in the Packages tab.

If the units genuinely have no source on disk — for example, they are from a compiled BPL you have integrated — then they are correctly identified as an external component and you should fill in the metadata: your company name as supplier, the appropriate license, and the version.

1.8.1. Your Application as a Component

The application itself always appears as the top-level component in the SBOM metadata section, not in the components list. This is set from the project name and version. The components list contains only the dependencies — the libraries and frameworks your application uses, not the application itself.

1.9. The AppCode Tab

The **AppCode** tab lists the internal modules identified during detection — the units that were found under your declared root folders and classified as belonging to your application.

This tab is informational. It allows you to verify that the root folder configuration is correct. If you see third-party library units appearing in the AppCode tab, their source files are under one of your declared root folders — either because you have included third-party source in your source tree, or because a root folder path is too broad. If you see your own units appearing in the Packages tab instead of here, a root folder is missing or incorrectly specified.

1.10. Generating the SBOM

Once all components have complete metadata, click **Generate SBOM**.

SBOMgen produces a CycloneDX 1.6 JSON file in the output folder for your project. The file is named with a timestamp: `2026-03-25_182040.sbom.json`.

The generated file contains a metadata section with the application name, version, author, generation timestamp, and tool information; a components array with one entry per detected external component; and a dependencies section linking the application to its components.

1.10.1. File Hashes

Where SBOMgen can locate a BPL file for a component (from the Delphi registry Known Packages entries), it computes a SHA-256 hash of that file and includes it in the SBOM. This provides a cryptographic fingerprint that can be used to verify the component has not been tampered with.

Components sourced from the GetIt CatalogRepository are only hashed when a `Source` subfolder is present in the CatalogRepository entry. Some packages — particularly those installed via GetIt on newer Delphi versions — do not include source, so no hash will be generated. This is expected behaviour, not an error.

1.10.2. Log Files

The detection and generation log is shown in the log panel. To save it, use **Save Log** — the dialog defaults to the project folder. Log files saved there are not managed by SBOMgen and accumulate over time; housekeeping is your responsibility.

1.11. Validating the SBOM

SBOMgen can validate the generated SBOM against the official CycloneDX 1.6 schema using the CycloneDX CLI tool. Validation catches structural errors and license identifier problems before the SBOM is delivered.

1.11.1. Installing the CycloneDX CLI

The CycloneDX CLI is a standalone executable:

1. Go to https://github.com/CycloneDX/cyclonedx-cli/releases
2. Download `cyclonedx-win-x64.exe` from the assets list
3. Rename it to `cyclonedx-cli.exe` and place it in a folder of your choice (e.g. `C:\Tools\`)
4. If Windows blocks the file, right-click → Properties → Unblock

This is a standalone binary — no installer and no .NET runtime required.

1.11.2. Configuring the CLI Path

In SBOMgen's **User Profile** tab, set the **CycloneDX CLI Path** to the full path of the downloaded executable. Click **Save Profile** after entering the path.

1.11.3. Running Validation

After generating an SBOM, click **Validate SBOM**. SBOMgen will run the CLI tool and display the result in the log panel. A passing validation confirms that the SBOM is structurally correct and uses valid SPDX license identifiers.

You can also validate independently at any time using the command line:

```
cyclonedx-cli.exe validate --input-file "MyApp.sbom.json" ^
  --input-format json --input-version v1_6 --fail-on-errors
```

1.11.4. Online Validation

For ad-hoc verification, the CycloneDX web tool at https://cyclonedx.github.io/cyclonedx-web-tool validates SBOM files directly in the browser without transmitting data to external servers. The NTIA compliance checker at https://sbomgenerator.com also validates CycloneDX 1.6 and checks against NTIA minimum element requirements.

1.11.5. Common Validation Errors

`Value should match one of the values specified by the enum` — A license identifier is not in the SPDX license list. Check the license field for the flagged component. For proprietary licenses, use `LicenseRef-Proprietary` rather than a plain text description.

`Required properties ["expression"] are not present` — A compound license expression (containing `OR`, `AND`, or `WITH`) or a `LicenseRef-` value has been placed in the `id` field instead of the `expression` field. This is handled automatically by SBOMgen for entries in the built-in catalog — if you see this error, check the license value you entered for a custom component.

`Required properties ["name"] are not present` — A license entry is missing required fields. This typically indicates a structural issue in a catalog entry.

1.12. The Metadata Catalog

The metadata catalog is the heart of SBOMgen's ability to produce complete SBOMs. It is a JSON file stored in your user profile that maps component names to their supplier, license, version, and unit membership information.

1.12.1. Built-in Entries

SBOMgen ships with a built-in catalog covering the most common Delphi components, including:

- Delphi RTL, VCL, FireDAC, XML, SOAP, Data Access, Windows API (all Embarcadero)
- Indy — bundled with Delphi, Mozilla Public License 1.1
- KSVC / Konopka Signature VCL Controls — bundled with RAD Studio
- Spring4D — Apache 2.0
- SynEdit — MPL 1.1
- VirtualTreeView — MPL 1.1 / LGPL
- DUnitX — Apache 2.0
- FastMM4 — MPL 1.1
- JCL and JVCL — MPL 1.1
- Fundamentals5 — BSD 2-Clause
- SVGIconImageList with bundled HtmlViewer — Apache 2.0
- And others

1.12.2. Catalog Updates

The built-in catalog is versioned. When SBOMgen is updated with a new catalog version, it merges the new entries into your saved catalog on next launch. Entries you have manually edited are preserved unchanged — the merge only replaces entries you have not touched.

1.12.3. Unit Membership

Some packages are identified by explicit unit membership lists. SVGIconImageList, for example, bundles HtmlViewer and MarkdownViewerComponents as source — units from those bundled libraries are attributed to SVGIconImageList because the catalog declares them as members. This is why a unit like `HtmlView` or `MarkdownProcessor` is correctly attributed to SVGIconImageList rather than appearing as an unknown component.

1.12.4. Prefix-Based Resolution

Large packages like Indy ship hundreds of units following a naming convention — `IdHTTP`, `IdTCPClient`, and so on. Rather than listing every unit explicitly, the catalog uses a prefix rule: any unit whose name starts with `Id` (case-sensitive) is attributed to Indy. Similarly, Konopka VCL Controls units start with `Rz`, and SynEdit units start with `Syn`.

1.12.5. Adding Prefix Rules for Custom Components

If you have a component whose units all share a common prefix, you can add a prefix rule so that all units are attributed automatically without listing every unit name individually.

To add or edit prefix rules for a component:

1. Select the component in the Packages tab.
2. Click the ... button next to the **Prefixes** field in the editor panel.
3. In the Prefix Editor dialog, type a prefix in the entry field and click **Add**, or press Enter.
4. Repeat for each prefix the component uses.
5. To remove a prefix, select it in the list and click **Delete**.
6. Click **OK** to save. The changes are written to the global metadata catalog immediately.

The prefix match is case-sensitive. A unit whose name starts with the registered prefix will be attributed to this component. Choose prefixes that are specific enough to avoid false matches — for example, Id correctly attributes all Indy units (`IdHTTP`, `IdTCPClient`) without matching unrelated units.

Current prefixes for the selected component are shown as a comma-separated list in the **Prefixes** field. If no prefixes are registered the field is empty.

Advanced use: Prefix rules can also be added by editing the catalog JSON file directly. The catalog file path is shown in the User Profile tab. Add a `prefixes array` to the relevant package entry — for example `"prefixes": ["Mc"]`. Save the file and restart SBOMgen to pick up the change. This approach is useful for bulk edits or scripted catalog maintenance.

1.12.6. Adding Custom Catalog Entries

When you edit a component's metadata in the Packages tab and click Apply, the entry is saved to your global catalog and will be available in all future projects.

1.13. Managing Projects

SBOMgen stores project configurations in your user documents folder. Each project remembers its name, Delphi version, root folders, MAP file path, DPR file path, and output folder.

1.13.1. Switching Projects

Select a different project from the project list. SBOMgen will load that project's configuration.

1.13.2. Multiple Projects

You can maintain as many projects as needed — one per application, or separate projects for different build configurations or compiler versions of the same application.

1.13.3. Project Output Folder

Each project writes its SBOM files to a dedicated output folder. SBOM files are named with a timestamp and accumulate over time — SBOMgen does not delete old files, so housekeeping is your responsibility.

1.14. Related Tools

1.14.1. DX.Comply

DX.Comply by Olaf Monien is another open-source Delphi SBOM tool. It operates as a RAD Studio IDE plugin and command-line tool, automatically triggering a MAP-file build and resolving every linked unit individually to its DCU or BPL file with a SHA-256 hash. It supports CycloneDX 1.5, SPDX 2.3, and produces human-readable HTML and Markdown compliance reports.

SBOMgen and DX.Comply are complementary rather than competing. DX.Comply answers "what is in my binary?" at unit granularity. SBOMgen answers "what packages is my binary composed of, and here is the properly attributed metadata for each one." For workflows that require curated supplier and license information at the package level — particularly for compliance documentation — SBOMgen's metadata editing and catalog persistence add value that automated unit-level scanning alone does not provide.

1.15. Troubleshooting

1.15.1. My Delphi version is not detected

SBOMgen reads the Windows registry to detect installed Delphi versions. If your version does not appear:

- Confirm Delphi has been run at least once since installation to write its registry entries
- Check under `HKEY_CURRENT_USER\Software\Embarcadero\BDS\` or `HKEY_LOCAL_MACHINE\Software\Embarcadero\BDS\` (or the equivalent `Wow6432Node` path on 64-bit Windows)
- Delphi 2007 and earlier use `\Borland\BDS\` or `\Borland\Delphi\` paths — these are supported
- Non-default installation paths (e.g. `C:\Programs\` instead of `C:\Program Files (x86)\`) are handled correctly since paths are read from the registry rather than assumed

1.15.2. My application appears as an unknown component

Your application's own units are being classified as external. Check that all source folders are declared as root folders. Root folder paths must point to the directories containing your `.pas` files, not parent directories. If your application spans multiple source trees — a common case when shared utility code is in a sibling folder — all trees must be listed. After updating root folders, click Detect again.

1.15.3. A third-party component appears as unknown

The component is not in the built-in catalog and has not been added as a custom entry. Select it in the Packages tab, fill in the supplier, license, and version, and click Apply. The entry will be saved for future use. If the component has many units sharing a common prefix, consider adding a prefix rule as described in Adding Prefix Rules for Custom Components.

1.15.4. The License dropdown is empty for a component

The stored license value is not a recognised SPDX identifier. Select a valid license from the dropdown or type a `LicenseRef-` expression. Do not leave the field blank.

1.15.5. The CatalogRepository is not found

For Delphi XE4 and earlier, the GetIt CatalogRepository did not exist — this message is expected and can be ignored. SBOMgen will still detect components from the IDE library paths. For later versions, verify that the CatalogRepository folder exists under your machine's public documents directory in the Embarcadero Studio folder for the relevant Delphi version.

1.15.6. No hash is generated for a component

File hashes require SBOMgen to locate either a BPL file in the Delphi registry Known Packages entries or source files in the GetIt CatalogRepository. Some components — particularly those installed via GetIt on newer Delphi versions — do not have a Source subfolder in the CatalogRepository, so no hash can be computed. This is expected behaviour.

1.15.7. Validation fails with license errors

Review the license values for each component. Common issues are plain text descriptions instead of SPDX identifiers, and compound expressions (`OR`, `AND`, `WITH`) that were entered manually in the wrong field. The full SPDX license list is at https://spdx.org/licenses. See LicenseRef- Expressions for guidance on proprietary licenses.

1.15.8. The MAP file contains no modules

Ensure the project was built with **Segments** or **Detailed** MAP file generation enabled (Project Options → Delphi Compiler → Linking → Map file). The default setting produces no useful module information.

SBOMgen is developed by William Meyer. For source code and updates, see the project repository.

SBOMgen generates CycloneDX 1.6 (ECMA-424, 1st Edition) output, the current version of the CycloneDX specification, validated against the official schema. Version 1.7 support is planned for a future release.